

Socialize.Video

BUILT IN A SESSION. BUILT TO LAST.

Socialize Video LLC
13 E 450 S, Midway UT 84049
jeff@socialize.video

TrustOffice Platform Briefing

Prepared for: Meeting with Jake

Date: July 15, 2026

From: Jeff Kohler, Socialize Video / TrustOffice

TrustOffice Platform Briefing

Prepared for: Meeting with Jake

Date: July 15, 2026

From: Jeff Kohler, Socialize Video / TrustOffice

1. Infrastructure Overview

TrustOffice is an AI-powered trust governance platform that runs entirely on infrastructure we control. No shared SaaS backends, no multi-tenant black boxes, every component is ours, deployed from our own repositories, and auditable end to end.

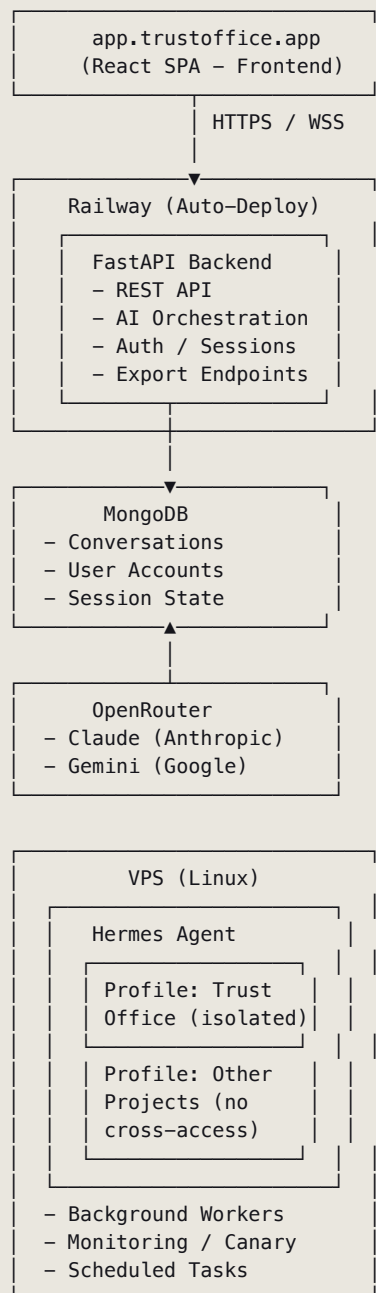
Current Architecture

The system is composed of six primary layers, each with a clear responsibility:

Layer	Technology	Role
Hosting	VPS (Linux)	Runs Hermes Agent, background workers, and supporting services
App Deployment	Railway	Auto-deploys the TrustOffice web app on every git push to main
Backend	FastAPI (Python)	REST API, AI orchestration, business logic, export endpoints
Frontend	React (SPA)	User-facing web chat portal at app.trustoffice.app
Database	MongoDB	Conversation history, user accounts, session state, metadata
AI Layer	OpenRouter	Routes to Claude and Gemini models depending on task

Hermes Agent sits alongside the app stack on the VPS and handles autonomous operations, monitoring, scheduled tasks, content generation, and maintenance workflows. It uses **profile-based isolation**, meaning each Hermes profile operates in its own context with its own skills, memories, and configuration. Profiles cannot see or access each other's data. This means TrustOffice's autonomous operations are completely walled off from any other project running on the same machine.

How the Pieces Connect



The flow, simply:

- 1 A user visits **app.trustoffice.app**, the React SPA loads in their browser.
- 2 They authenticate (JWT-based) and start a chat session.
- 3 Each message is sent to the **FastAPI backend** on Railway via HTTPS.

- 4 The backend processes the message, calls **OpenRouter** to route to the appropriate AI model (Claude for complex reasoning, Gemini for faster tasks).
- 5 The AI response is streamed back to the user and stored in **MongoDB** for conversation history.
- 6 **Hermes Agent** on the VPS runs independently, monitoring health, handling scheduled tasks, and performing autonomous operations within its isolated TrustOffice profile.

Railway auto-deploys mean every push to the main branch goes live within minutes. No manual deployment steps, no server SSH sessions. This keeps our release cycle fast and our deployment history fully auditable through git.

2. Security Environment

Security is handled at two levels: **external** (how the system protects itself from the internet) and **internal** (how the system protects data between its own components and users).

External Security

Control	Implementation	Status
Transport Encryption	Full TLS/HTTPS on all endpoints. No plaintext traffic accepted. SSL terminated at the Railway proxy layer.	 Live
CORS Policy	Cross-Origin Resource Sharing is locked to specific allowed origins (trustoffice.app and app.trustoffice.app). Requests from any other domain are rejected.	 Live
User Authentication	JWT (JSON Web Tokens) for all user sessions. Tokens are signed, time-limited, and validated on every API request.	 Live
Admin Access	Admin API key required for all privileged operations (user management, system config, data exports). Separate from user JWT auth.	 Live
Input Validation	All API endpoints validate and sanitize input via Pydantic models. No raw SQL or unvalidated payload acceptance.	 Live

Internal Security

Profile-Based Isolation (Hermes Agent)

Hermes Agent runs multiple profiles on the same VPS, but each profile is completely sandboxed:

- Each profile has its own **skills, memories, cron jobs, and configuration files** stored in separate directory trees.
- Profiles **cannot read, write, or access** another profile's data. There is no shared state, no shared database, no shared API context.
- The TrustOffice profile has no visibility into any other project running on the machine, and vice versa.

This means even if an attacker compromised one Hermes profile, they would have no path to TrustOffice data or operations through the agent layer.

Secrets Management

- API keys, database credentials, and service tokens are stored as **local environment variables** on the VPS and Railway, never in the codebase, never in git, never exposed to the frontend.
- Secrets are injected at runtime from the hosting platform's encrypted secret store.
- No secrets are logged, cached in plain text, or transmitted in API responses.

Information Control Policies

- User conversation data is stored in MongoDB and is **scoped per user account**. No user can access another user's conversations.
- Admin-level operations require the admin API key, user JWTs cannot escalate.
- All API responses are scoped to the authenticated user's own data. There is no global read access from the frontend.

3. Recent Performance Fixes

Two issues were identified and **fully resolved** in tonight's deployment. Both fixes are live in production now.

Issue 1: Slow Response Times (2-3 minutes per query)

What was happening: The FastAPI backend was making synchronous calls to the OpenRouter AI API. Because FastAPI runs on an async event loop, a synchronous HTTP call blocks the entire event loop, meaning no other request could be processed until the AI response came back. When multiple users were active, requests queued up and response times ballooned to 2-3 minutes.

What was fixed:

Fix	Description
<code>`asyncio.to_thread` wrapper</code>	AI API calls are now executed in a thread pool via <code>`asyncio.to_thread()`</code> , freeing the event loop to handle other concurrent requests while waiting for AI responses.
Parallelized intent classification	The intent classification step (which determines how to route a query) was previously sequential. It now runs in parallel with the main AI call, cutting total round-trip time significantly.
Reduced system prompt size	The system prompt sent with each request was trimmed from a large context block to a lean, focused version. Less token overhead means faster AI responses and lower API costs.

Result: Response times dropped from 2-3 minutes to **under 15-20 seconds** for typical queries.

 Deployed and live.

Issue 2: Mobile Browser Queries Failing on Navigation

What was happening: When a mobile user sent a query and then switched to another app or tab, the browser would background the page and kill the active network connection. When they returned, the query appeared to have failed with no way to recover.

What was fixed:

Fix	Description
Wake Lock API	The app now requests a screen wake lock during active AI queries, preventing the device from sleeping and dropping the connection mid-request.
`visibilitychange` handler	When the user navigates away and returns, the app detects the visibility change and checks whether the in-flight request is still valid or needs to be retried.
Auto-reconnection	If the connection drops during a query, the app automatically attempts to reconnect and resume, no manual refresh required.
Polling fallback	If WebSocket streaming fails, the app falls back to HTTP polling to retrieve the response, ensuring the user still gets their answer.

Result: Mobile users can now switch away from the tab and return without losing their query. The app recovers automatically.  Deployed and live.

4. Feature Roadmap

Below is an assessment of each feature Jake has asked about. Each is categorized by feasibility and estimated development effort.

4.1 File Upload / Download (Image Attachments in Chat)

Status: Quick Win (1-2 days)

Feasibility: Fully feasible. This is a well-understood pattern and the backend already has the API structure to support it.

Approach:

- Add an image upload endpoint to the FastAPI backend that accepts multipart form data.
- Store uploaded images in cloud object storage (AWS S3, Cloudflare R2, or similar). We'd likely use Cloudflare R2 for zero egress fees and alignment with our existing Cloudflare infrastructure.
- Attach image references to chat messages in MongoDB (store the image URL alongside the message text).
- Frontend: Add a file picker / drag-and-drop zone in the chat composer. Display image thumbnails inline in the conversation thread.
- Images would be served via signed, time-limited URLs for security, no public read access.

What's needed:

- Storage bucket setup (R2 or S3), 1-2 hours
- Backend upload endpoint + image attachment to messages, 4-6 hours
- Frontend file picker, preview, and display, 4-6 hours
- Security: signed URLs, file type validation, size limits, 1-2 hours

Total: ~1-2 days of focused development.

4.2 User Notifications / Messaging

Status: Medium Effort (1-2 weeks)

Feasibility: Feasible with two approach options depending on the use case.

Option A, In-App Notifications (WebSocket-based, real-time):

- Implement a WebSocket connection for each active user session.
- Server pushes notifications when relevant events occur (new report ready, task completed, etc.).
- Best for: active users who are currently in the app.
- Effort: ~3-5 days

Option B, Email Notifications (Polling + SMTP):

- Backend checks for events on a schedule and sends email notifications via an email service (Brevo, SendGrid, or similar).
- Best for: users who aren't actively in the app but need to know when something is ready.
- Effort: ~2-3 days

Recommended: Start with Option B (email notifications) for immediate value, then add WebSocket-based in-app notifications as Phase 2. This gives Jake notification capability quickly without a heavy real-time infrastructure build.

What's needed for email notifications:

- Email service integration (Brevo/SendGrid), 2 hours
- Notification trigger logic in backend, 1 day
- User notification preferences (opt-in/opt-out), 1 day
- Email templates, 1 day
- Testing, 1 day

Total: ~1 week for email notifications, ~2 weeks for full real-time in-app notifications.

4.3 CSV Report Download Links**Status: Quick Win (1-2 days)**

Feasibility: The backend already has data export infrastructure. CSV generation is straightforward, we can query MongoDB, format results as CSV, and serve via a download endpoint.

Approach:

- Add a `/reports/export` endpoint that accepts parameters (date range, report type, filters).

- Backend queries MongoDB, formats results as CSV using Python's built-in `csv` module or `pandas`.
- Serve the CSV as a file download with appropriate `Content-Type` and `Content-Disposition` headers.
- Optionally: generate the CSV asynchronously, store it, and provide a time-limited download link (better for large reports).

Existing capabilities:

- The backend already has export logic patterns in place for other data types.
- MongoDB aggregation pipelines are already used for data queries.
- JWT auth infrastructure means we can scope exports to the requesting user's data.

What's needed:

- CSV export endpoint + formatting, 4-6 hours
- Frontend download button / link, 2-3 hours
- Testing + edge cases (large datasets, empty results), 2-3 hours

Total: ~1-2 days.

4.4 PDF Generation + Download Links

Status: Medium Effort (1-2 weeks)

Feasibility: Fully feasible. PDF generation is more complex than CSV because it involves layout, styling, and potentially branded templates, but it's a well-trodden path.

Approach:

- Use a Python PDF generation library. Two options:
 - **WeasyPrint** (HTML/CSS → PDF): Best for branded, styled documents. We design the PDF as an HTML template with CSS, then render to PDF. Supports logos, tables, headers/footers.
 - **ReportLab**: More programmatic, finer control over layout, but more code to write.
- Recommended: **WeasyPrint**, it lets us design PDFs as HTML templates, which is faster and produces cleaner results with less code.
- Add a `/reports/pdf` endpoint that accepts the same parameters as CSV export but returns a styled PDF.

- PDFs are generated on-demand and served as downloads. For large or complex reports, generate asynchronously and provide a download link.

What's needed:

- PDF template design (HTML/CSS with TrustOffice branding), 1-2 days
- Backend PDF generation endpoint, 1 day
- Frontend download trigger, 2-3 hours
- Testing (layout, pagination, large reports), 1-2 days

Total: ~1 week for a polished, branded PDF export feature.

Roadmap Summary

Feature	Status	Estimated Effort
File upload / image attachments	Quick Win	1-2 days
User notifications (email)	Medium Effort	~1 week
User notifications (in-app real-time)	Medium Effort	~2 weeks
CSV report download links	Quick Win	1-2 days
PDF generation + download links	Medium Effort	~1 week

If we were to prioritize: CSV export and file upload are the fastest wins. Both could be live within a week. PDF generation and email notifications would follow as the next sprint. In-app real-time notifications would be the largest build and could come after.

5. E-Commerce Vision

Jake asked about how an e-commerce store would be built, how Hermes fits in, and what the reliability/privacy/security picture looks like. Here's the high-level vision.

Architecture Overview

The e-commerce store would be a **separate storefront** from the main TrustOffice app, a dedicated site (e.g., store.trustoffice.app) built for product browsing, checkout, and order management. It would integrate with TrustOffice's existing backend for shared user accounts and data.



Where Hermes Fits

Hermes Agent would serve as the **autonomous operations layer** for the store:

- **Inventory monitoring:** Hermes tracks stock levels and alerts when products need restocking. Can automate reordering through supplier APIs.
- **Order fulfillment:** When an order comes in via Stripe webhook, Hermes can trigger fulfillment workflows, generating shipping labels, notifying the fulfillment team, updating order status.

- **Customer communications:** Hermes can send order confirmations, shipping notifications, and follow-up emails, all through the same secure notification infrastructure described in Section 4.2.
- **Sales analytics:** Hermes can generate periodic sales reports, identify trends, and surface insights, all within its isolated profile.
- **Storefront management:** Hermes can manage product listings, update pricing, and handle promotional content through scheduled or on-demand tasks.

Reliability

- **Payment processing** goes through Stripe Checkout, Stripe handles all PCI compliance, card data, and payment security. Card numbers never touch our servers.
- **Webhook reliability:** Stripe webhooks are delivered with automatic retries. Our backend verifies webhook signatures to prevent spoofing.
- **Inventory consistency:** Orders and inventory are managed in MongoDB with atomic operations to prevent overselling.
- **Deployment:** The store frontend deploys on the same Railway auto-deploy pipeline as the main app, same fast, auditable release cycle.

Privacy & Security

The store inherits all of TrustOffice's existing security controls:

- **TLS/HTTPS** on all endpoints
- **JWT authentication** shared with the main app (single sign-on)
- **CORS locked** to store.trustoffice.app
- **Profile isolation:** Hermes's store operations run in a separate profile from the main TrustOffice agent profile, no cross-contamination of data or operations.
- **No card data storage:** Stripe handles all payment data. We store order metadata (amount, items, status) but never card numbers, CVVs, or payment tokens.
- **Customer data:** Order history and customer information are scoped to each user's account, same as conversation data in the main app.

Estimated Build Time

- **MVP storefront + Stripe Checkout:** 2-3 weeks

- **Hermes inventory/fulfillment integration:** 1-2 weeks (can run in parallel with storefront build)
- **Customer notification system:** 1 week (shared with the notification feature from Section 4.2)

Total: ~3-5 weeks for a fully functional e-commerce store with automated operations.

6. Monitoring & Reliability

Tonight's deployment didn't just fix bugs, it also added three new monitoring and reliability systems that are now running in production. These give us proactive visibility into the platform's health rather than waiting for a user to report a problem.

Synthetic Canary (AI Response Time Monitoring)

What it does: An automated probe hits the TrustOffice AI chat endpoint every 30 minutes and measures the full response time, from request to completed AI response.

Why it matters: If response times start creeping up (the exact issue we just fixed), we know within 30 minutes, not when a user complains. The canary logs every check and alerts if response time exceeds a threshold.

Status:  Live and running.

Post-Deploy Smoke Test

What it does: After every deployment to Railway, an automated test suite runs **9 endpoint checks** against the production API:

-  Health check endpoint
-  User authentication (login)
-  Chat message send
-  Chat message retrieval
-  Conversation history
-  User profile
-  Admin endpoint (with API key)
-  Export endpoint
-  Static asset serving

Why it matters: If a deployment breaks something, we know immediately, not hours later when a user hits the broken endpoint. If any check fails, the deploy is flagged for review.

Status:  Live and running on every deploy.

In-App Error Reporting

What it does: Two complementary error capture systems:

- **Frontend crash capture:** If the React SPA encounters an unhandled error or exception, it's automatically captured and logged before the user sees a generic error message. This includes the error stack trace, browser info, and the action that triggered it.
- **Backend error logging:** All unhandled exceptions in the FastAPI backend are logged with full context, request details, user ID (if available), stack trace, and timestamp.

Why it matters: We see errors as they happen, with enough context to fix them, not vague "something broke" reports. Errors are centralized and can be reviewed in real time.

Status:  Live and capturing.

Weekly Audit (New Checks)

A weekly automated audit now runs **5 additional checks** on top of the continuous monitoring:

Check	What It Verifies
SSL/TLS certificate health	Certificates are valid and not approaching expiration
Database connection integrity	MongoDB is reachable, responsive, and not showing connection pool exhaustion
API endpoint availability	All 9 critical endpoints respond within acceptable latency
AI provider health	OpenRouter is reachable and returning responses within expected timeframes
Deployment freshness	The production deployment matches the latest git commit on main (no stale deploys)

Status:  Scheduled and running weekly.

Summary

Topic	Key Takeaway
Infrastructure	Fully owned stack, VPS + Railway + FastAPI + React + MongoDB + OpenRouter + Hermes. No shared SaaS dependencies. Profile-isolated agent operations.
Security	TLS, CORS, JWT, admin API key externally. Profile isolation, local secrets, per-user data scoping internally.
Performance	Two critical issues (slow responses, mobile drops) are fixed and live. Response times down from 2-3 min to 15-20 sec. Mobile queries survive backgrounding.
Features	CSV export and file upload are quick wins (1-2 days each). PDF generation and email notifications are ~1 week each. Real-time in-app notifications are ~2 weeks.
E-Commerce	Separate storefront with Stripe Checkout, Hermes-managed operations, full security inheritance. ~3-5 week build for MVP.
Monitoring	Three new systems live: synthetic canary (30-min response monitoring), post-deploy smoke tests (9 checks), in-app error reporting. Weekly audit with 5 health checks.

Prepared by TrustOffice / Socialize Video

July 14, 2026